



Cycle-Accurate HSA Simulator

Chien-Chih Chen

Advisor: Tien-Fu Chen

SoC & ESW Lab

國立交通大學

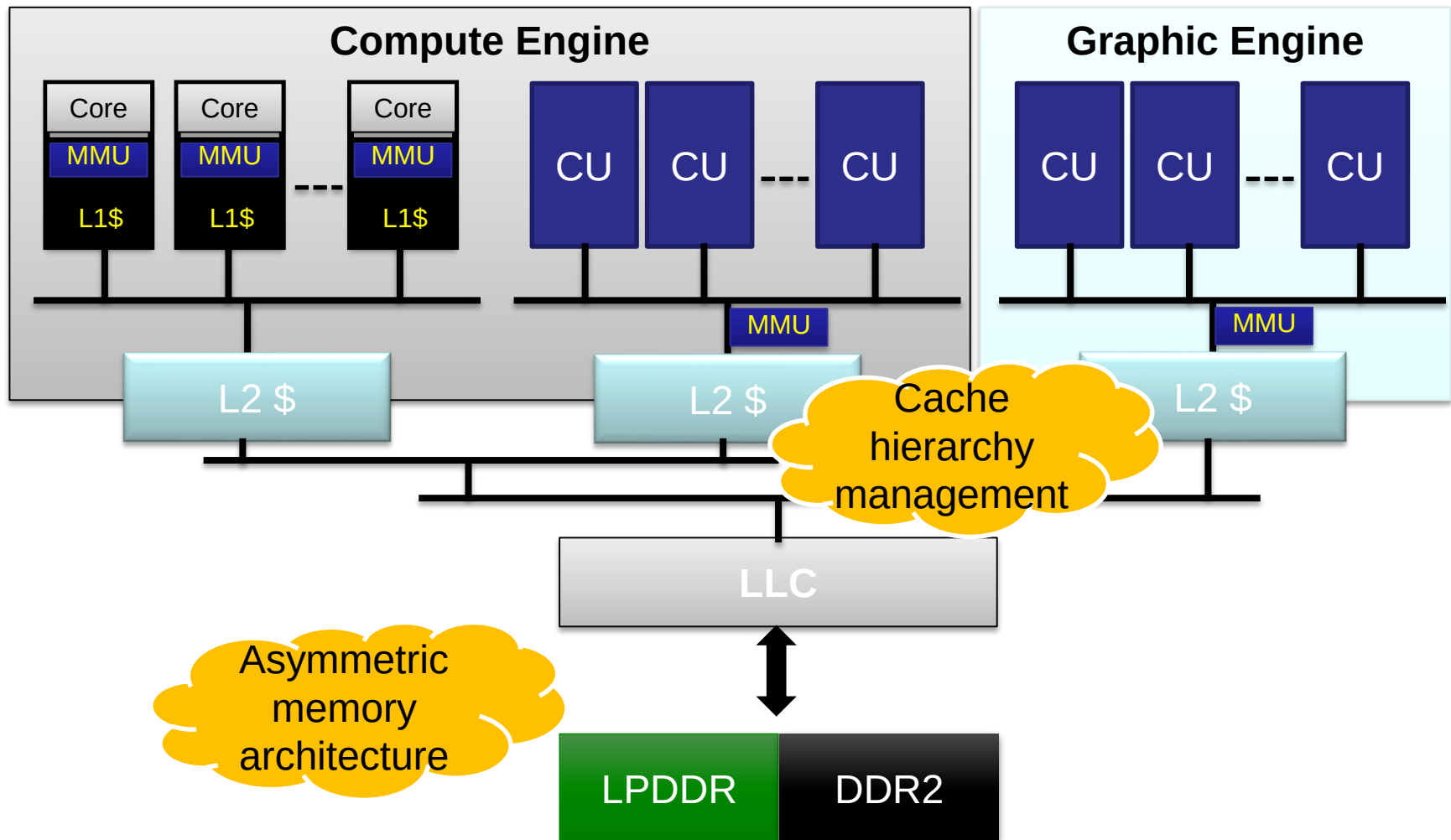
National Chiao Tung University

2015/5/18

Outline

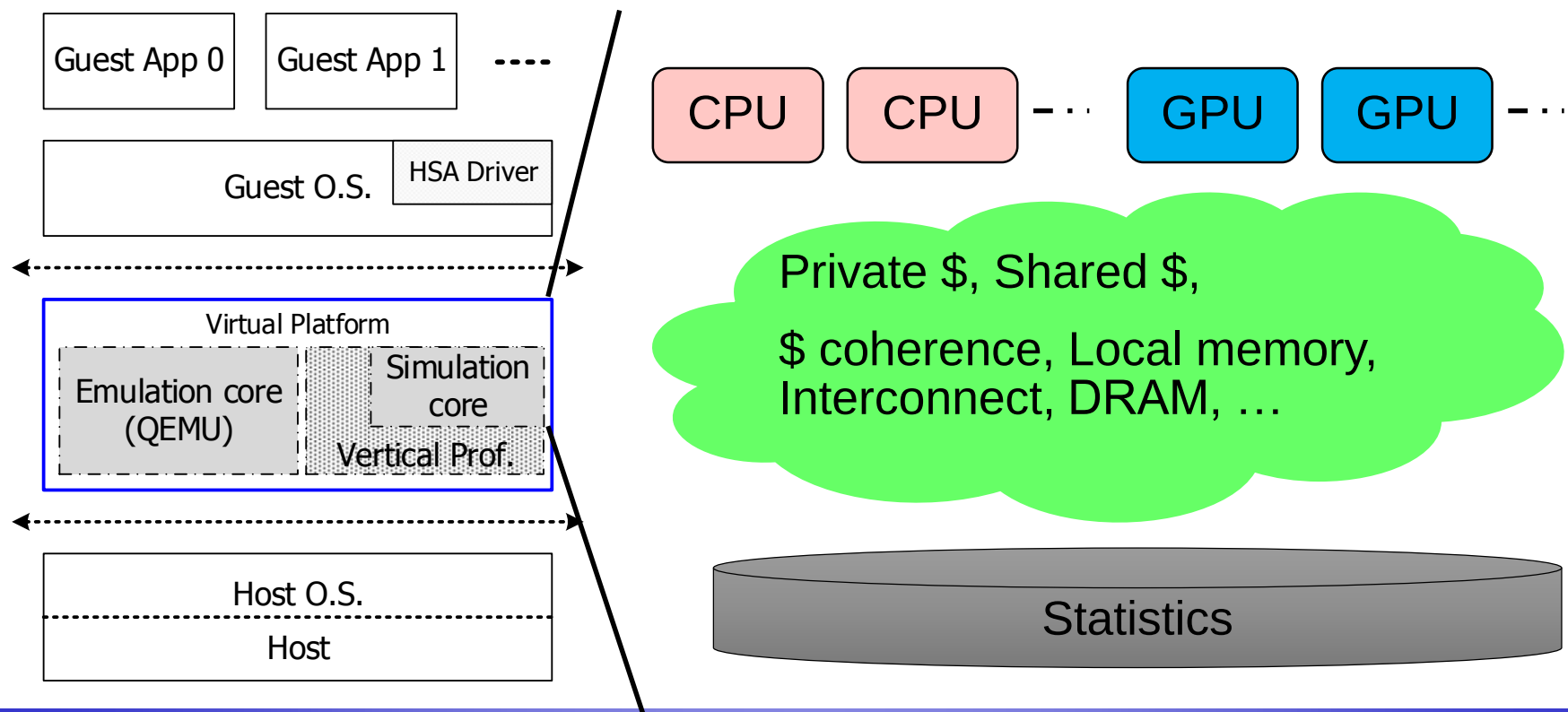
- Why do we need a fast cycle-accurate HSA simulator
- Basics of SW simulator
- Our event-driven architectural simulator w/ QEMU
- Our fast cycle-accurate HSA simulator

Heterogeneous multicore issues



We need a simulator

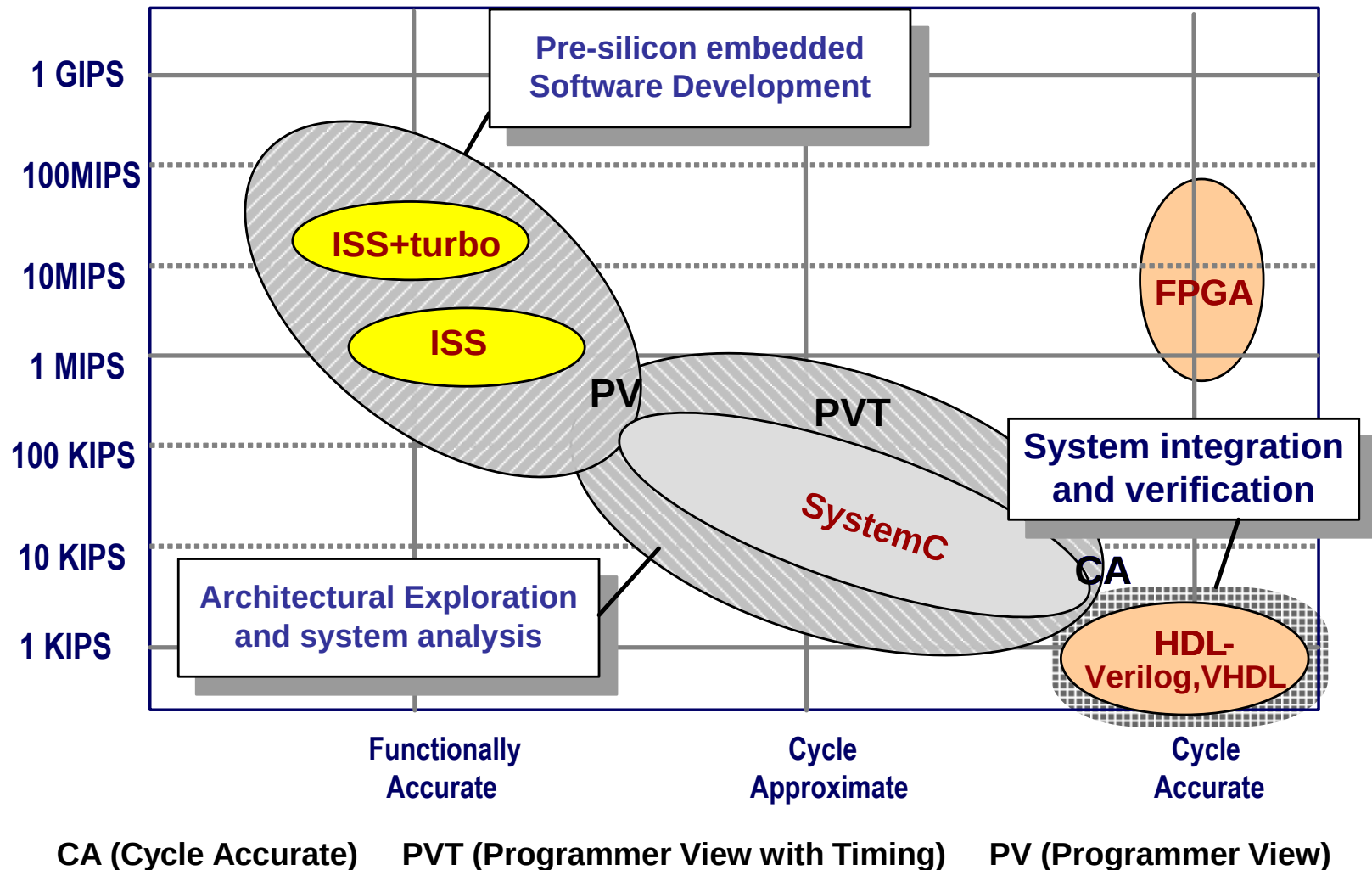
- Simulator is models computer devices (or components) to predict outputs and performance metrics on a given input.
- Used for three aspects
 - ⊙ Application development, System verification, System architecture exploration, System performance evaluation



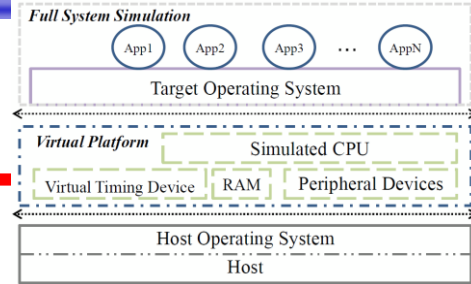
Different Abstraction Levels

Abstraction Level	Attributes						
	Simulation Throughput	CPU	Cycle Accurate	Bus model	Software	Target OS	MPEG4 704x576 (4CIFx60f)
Software Only	1~2 GIPS	None	No	No	C, C++	none	30 s
ISS simulation	1~10 MIPS	ISS	Maybe	possible	C, C++ assembly	possible	5 hr
SystemC simulation	10~100 KIPS	cycle-accurate model	Maybe	TLM/ timed model	C, C++ assembly	almost not practical	18 hr
HDL Emulation	10~1000 IPS	HDL	Yes	Yes	assembly	Not Practical	1200 days
FPGA Emulation	50~100 MIPS	Real or hard core	Yes	Yes	C, C++ assembly	real	10 m
Silicon Prototype	Actual						

Abstraction and Accuracy



Categories of SW Simulators



- Scope: micro-architecture vs. full-system
 - ⊙ Only one microprocessor or the whole computer system.
- Detail : functional vs. timing (or performance) simulators.
 - ⊙ Functional simulators emphasize achieving the same function as the modeled components (**what is done**)
 - ⊙ Timing simulators strive to accurately reproduce the performance/timing features (**when is it done**) of the targets in addition to their functionalities.
- Input : trace-driven vs. execution-driven simulators.
 - ⊙ Traces are **pre-recorded** streams of instructions with some fixed input.
 - ⊙ Execution-driven simulators allow **dynamic** change of instructions to be executed depending on different input data.

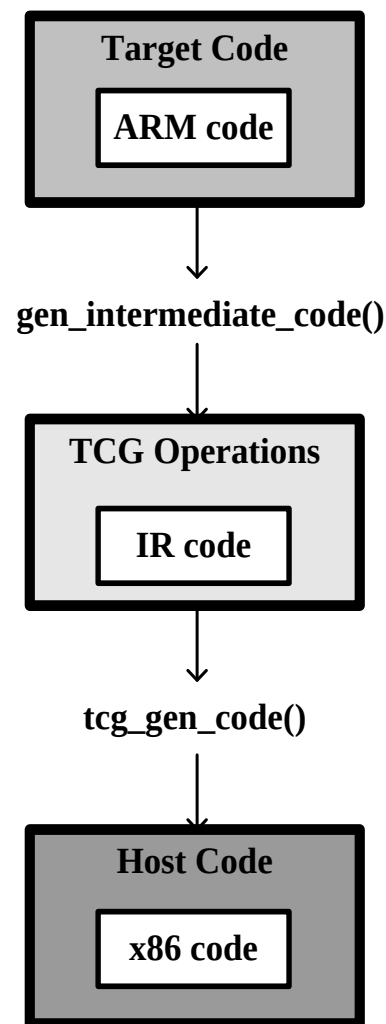
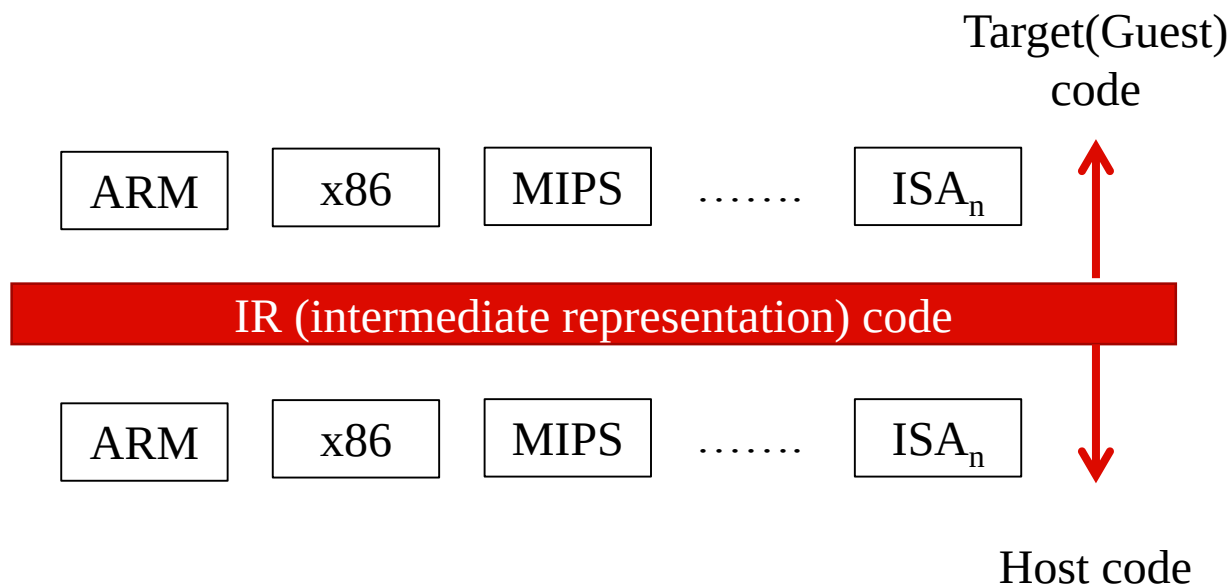
QEMU

A full-system functional simulator

- What is the Qemu?
 - ⊙ A generic and open source machine emulator and virtualization machine.
- Feature
 - ⊙ Simulation speed is fast!!! (DBT)
 - ⊙ Support many ISAs
 - ⊙ Portable : can run on different machines
 - ⊙ Two mode: user-level simulation & Full-system simulation
 - ⊙ Open source

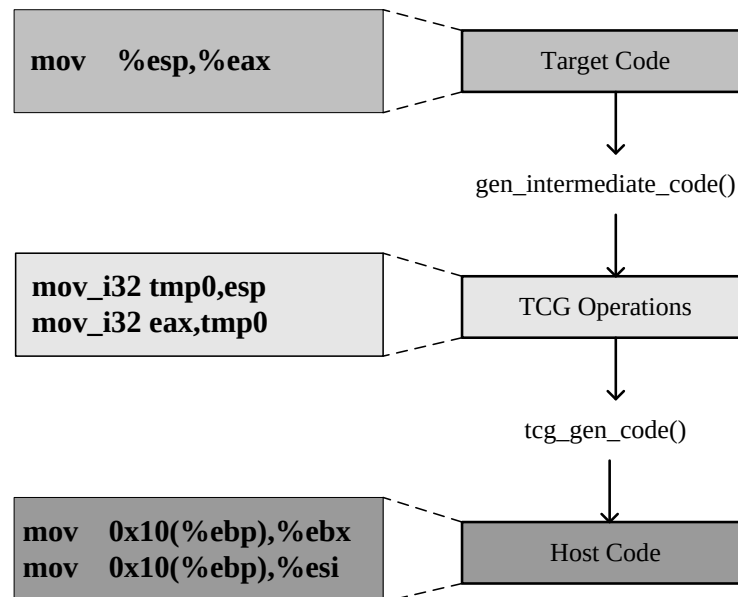
How to support many ISAs on different machines?

- **TCG (Tiny Code Generator)**
 - ⊙ A generic backend for a C compiler.



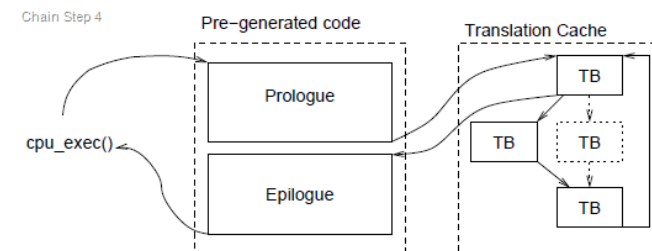
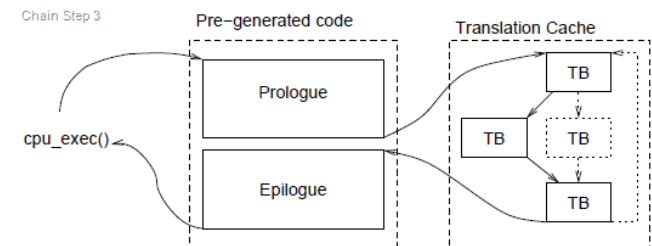
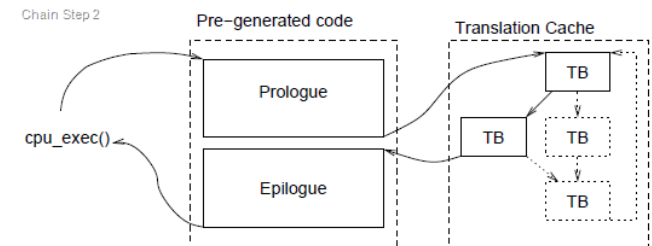
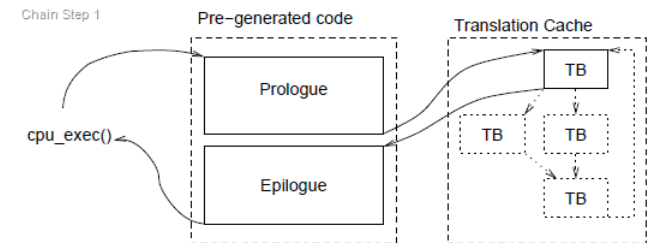
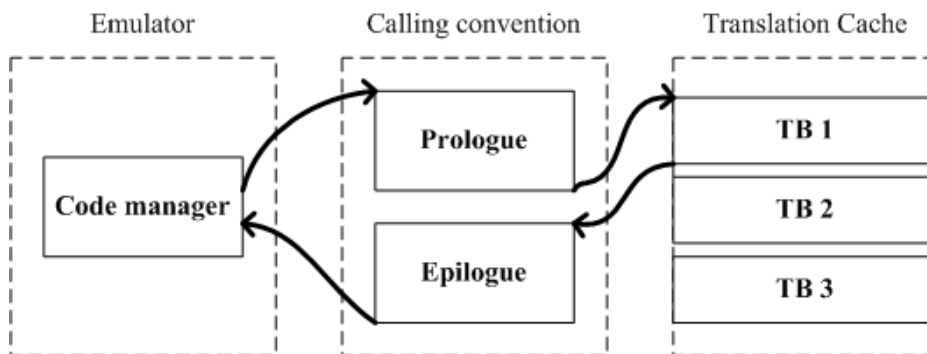
How to translate the ISA_1 to ISA_2 ?

- DBT (Dynamic binary translation)
 - ⊙ Looks at a short sequence of code (usually a basic block) then translates it and caches the resulting sequence.
- TB (translation block)
 - ⊙ The translation unit in Qemu. (as a basic block)



Acceleration during translated code

- Code cache
 - ⊙ Store every translated code in cache and reuse it.
- Chain
 - ⊙ Link one TB to next TB reduce trips to the manger.



CMP\$im

A trace-driven timing simulator

“CMP\$im: A Pin-based on-the-fly multi-core cache simulator”

[A. Jaleel et al, 2008 MOBS]

- Feature

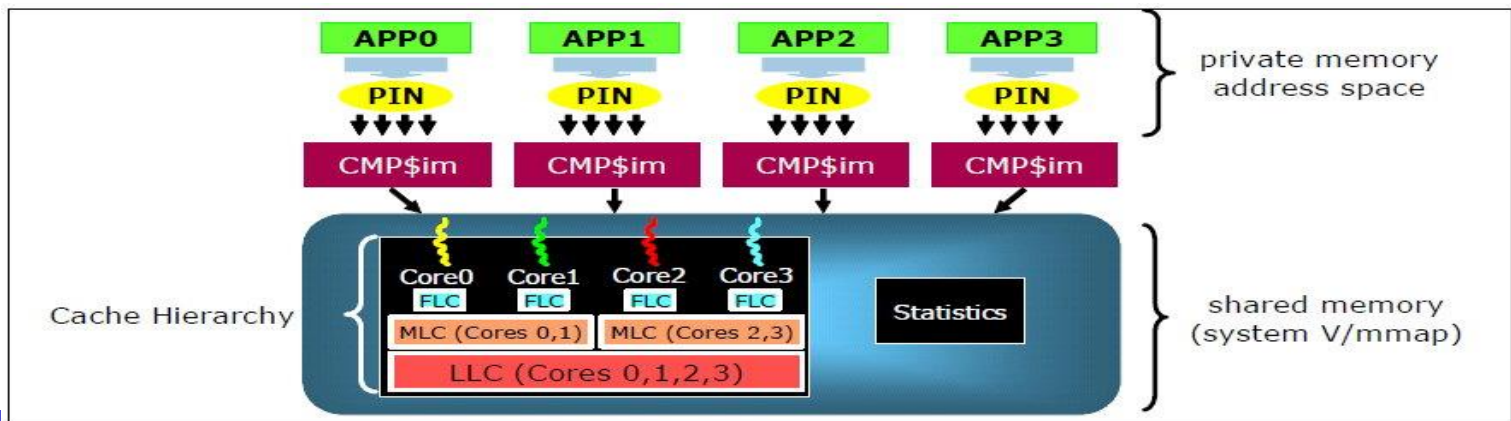
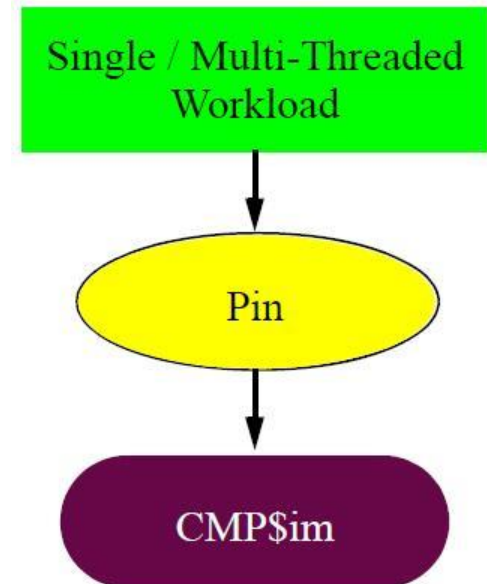
- Use PIN to conduct exploratory memory performance studies.

- Pros

- Native execution, faster speed
- Flexible memory module

- Cons

- Limit on host ISA.
- Only can use on application level based simulation.
- Only used on memory studies
- Low-scalability



Multi2Sim

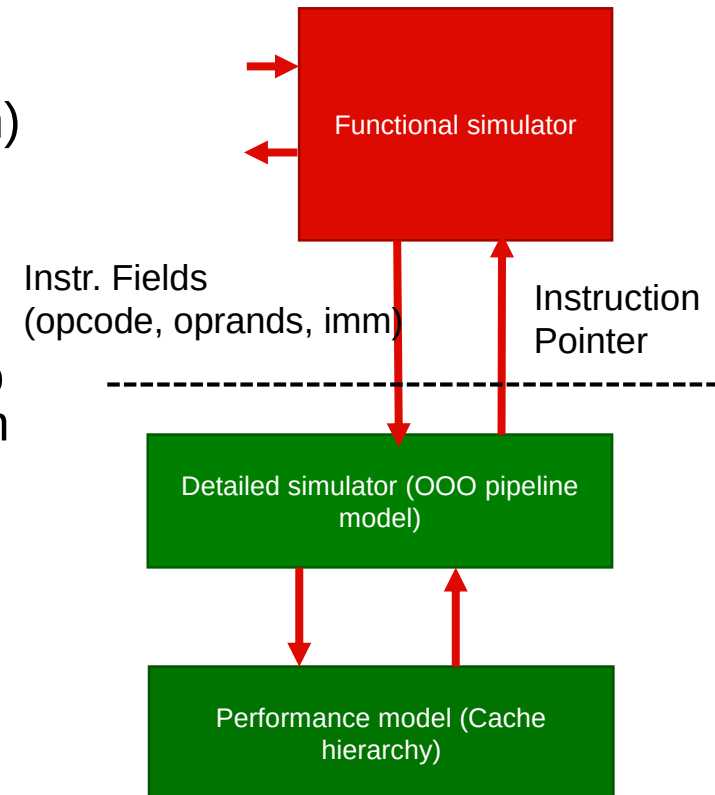
A user-level timing simulator

What is Multi2Sim

- A simulation framework for CPU-GPU heterogeneous computing
 - ⊙ CPU: X86
 - ⊙ GPU (OpenCL): Southern-Island
- Application-only emulation
- It provides models for
 - ⊙ Superscalar, multithreaded, and multicore CPUs, and GPU architectures
 - ⊙ Memory hierarchies

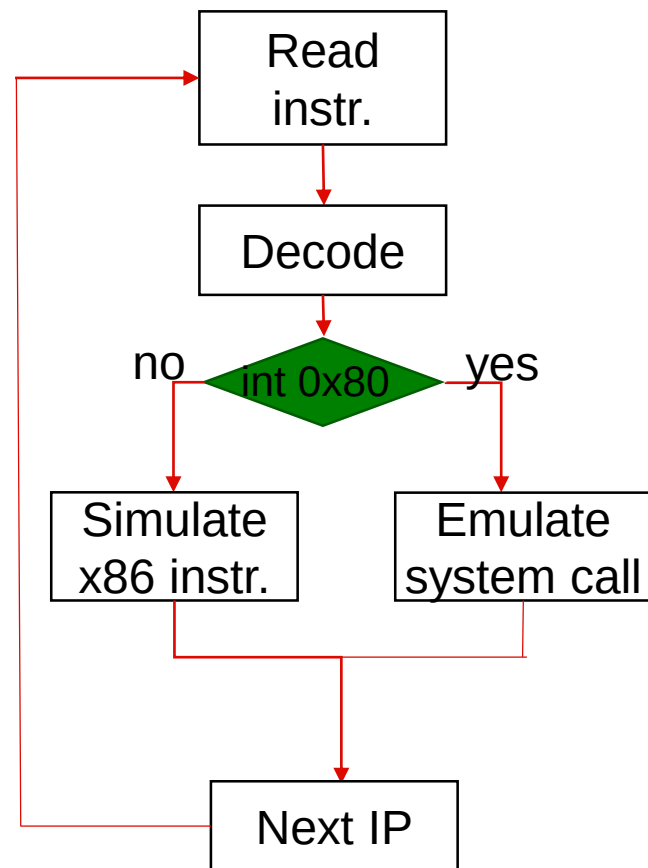
Multi2Sim simulation flow

- Three main components
 - ⊙ Functional simulator (emulator)
 - ⊙ Detailed simulator (pipeline simulation)
 - ⊙ Performance model (Cache hierarchy, prefetch)
- Functional simulator
 - ⊙ Map the guest execution instruction to the implementation function which can run on the host
- Detailed simulator
 - ⊙ Calculate timing (data hazard, control hazard)
- Performance model
 - ⊙ Calculate timing (structural hazard)



Instruction emulation in simulator

- Emulation of x86 instructions
 - ⦿ Update x86 registers.
 - ⦿ Update memory
- Emulation of Linux system calls
 - ⦿ Instruction “int 0x80” is used to perform a system call
 - ⦿ Emulate the system call according to “system call code” and its arguments.

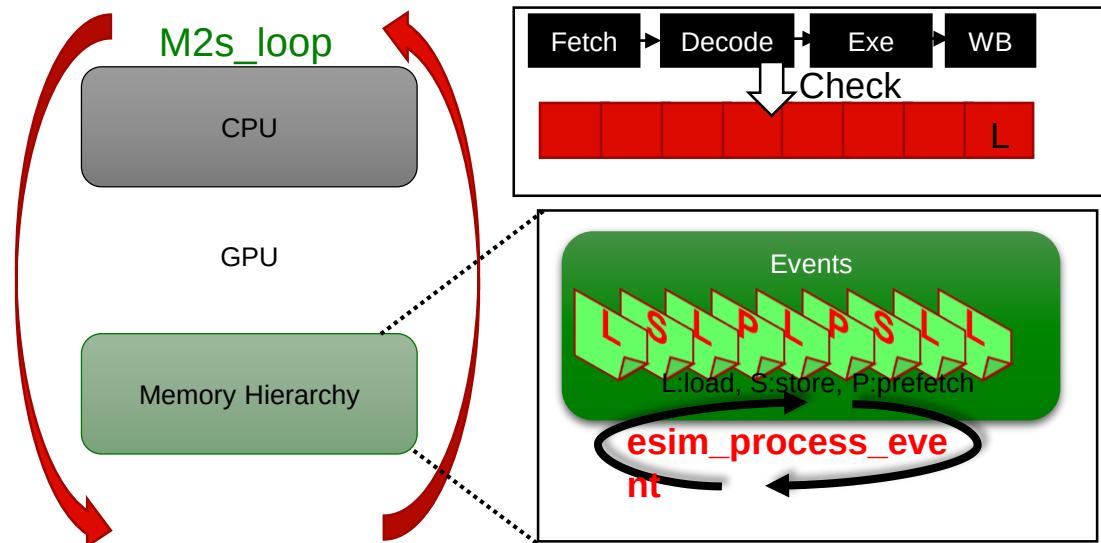


The memory hierarchy

- Flexible hierarchies
 - ⊙ Any number of caches organized in any number of levels.
 - ⊙ Each architecture with a timing simulation specifies its own **entry point**(cache memory) in the memory hierarchy, for data or instructions.
 - ⊙ Cache coherence is guaranteed with an implementation of the 5-state **MOESI protocol**.
- Event queue support
 - ⊙ Only simulate when the simulation time arrive the event trigger time

Interface between CPU & Mem. Hierarchy

- `mod_access` (return: `event_id`)
 - for creating an event on Memory hierarchy
 - `ACCESS_MODULE`, `ACCESS_TYPE`, `ADDRESS`
- `mod_in_flight_access`
 - Check event finished or not
 - `ACCESS_MODULE`, `EVENT_ID`, `ADDRESS`

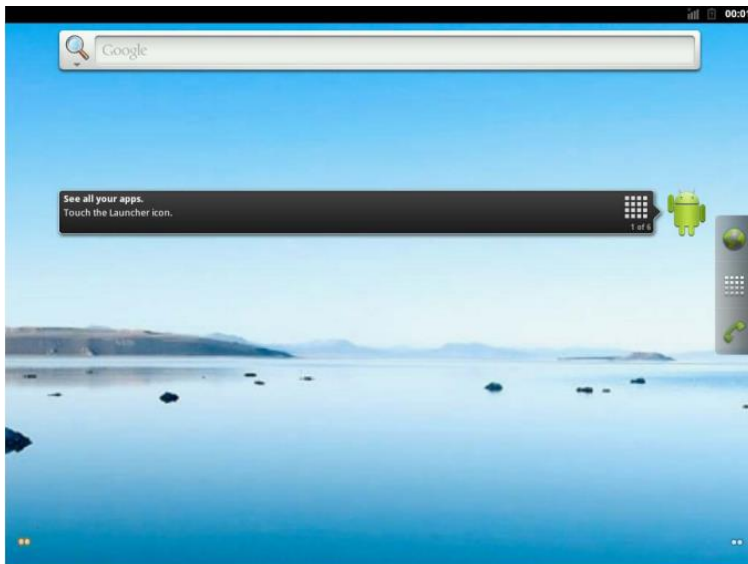


Gem5

A full-system timing simulator

What is Gem5

- Integrate two simulators
 - ⊙ M5: CPU models, ISAs, I/O devices, infrastructure
 - ⊙ Gems (Ruby): cache coherence protocols, interconnect model



```
mycpu-SC2B0617
root@(none) / # cat /proc/cpuinfo | grep processor | egrep 6, ; uname -a
processor       : 60
processor       : 61
processor       : 62
processor       : 63
Linux (none) 2.6.28-rc4-dirty #5 SMP Mon Jun 21 13:39:45 PDT 2010 x86_64 GNU/Linux
root@(none) / #

mycpu-SC2B0617
0: system.remote_gdb.listener: listening for remote gdb on port 7057
0: system.remote_gdb.listener: listening for remote gdb on port 7058
0: system.remote_gdb.listener: listening for remote gdb on port 7059
0: system.remote_gdb.listener: listening for remote gdb on port 7060
0: system.remote_gdb.listener: listening for remote gdb on port 7061
0: system.remote_gdb.listener: listening for remote gdb on port 7062
0: system.remote_gdb.listener: listening for remote gdb on port 7063
**** REAL SIMULATION ****
info: Entering event queue @ 5522591772000. Starting simulation...
warn: Don't know what interrupt to clear for console.
warn: instruction 'fncw_Mw' unimplemented
warn: instruction 'fldcw_Mw' unimplemented
warn: Tried to clear PCI interrupt 14
```

Features

- Execution modes: System-call Emulation (SE) & Full-System (FS)
- ISAs: Alpha, ARM, MIPS, Power, SPARC, x86
- CPU models: AtomicSimple, TimingSimple, InOrder, and O3
- Cache coherence protocols: broadcast-based, directories, etc.
- Interconnection networks: Simple & Garnet (Princeton, MIT)
- Devices: NICs, IDE controller, etc.
- Multiple systems: communicate over TCP/IP

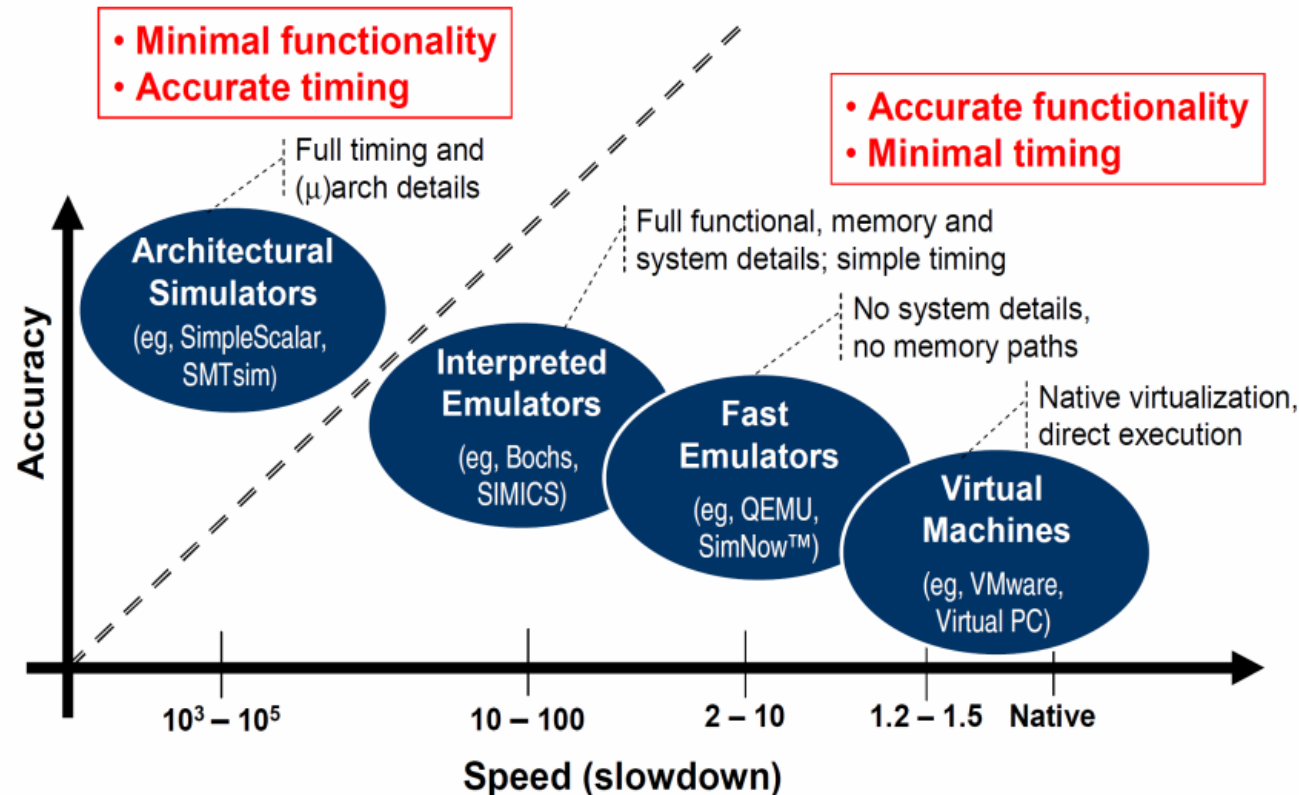
Interpretation simulation

mov_rm8_imm8



```
void x86_isa_mov_rm8_imm8_impl(X86Context *ctx)
{
    unsigned char value = ctx->inst.imm.b;
    X86ContextStoreRm8(ctx, value);
    x86_uinst_new(ctx, x86_uinst_move, 0, 0, 0, x86_dep_rm8, 0, 0, 0);
}
```

Speed and Accuracy of ISS



[A. Falcon et al, ISPASS, 2007]

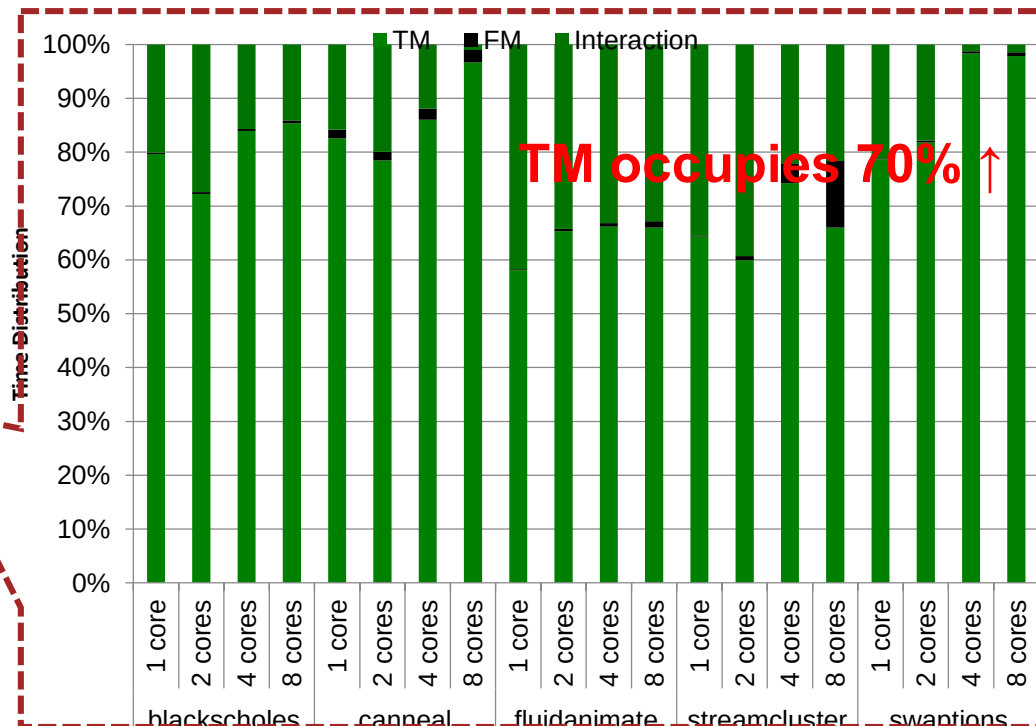
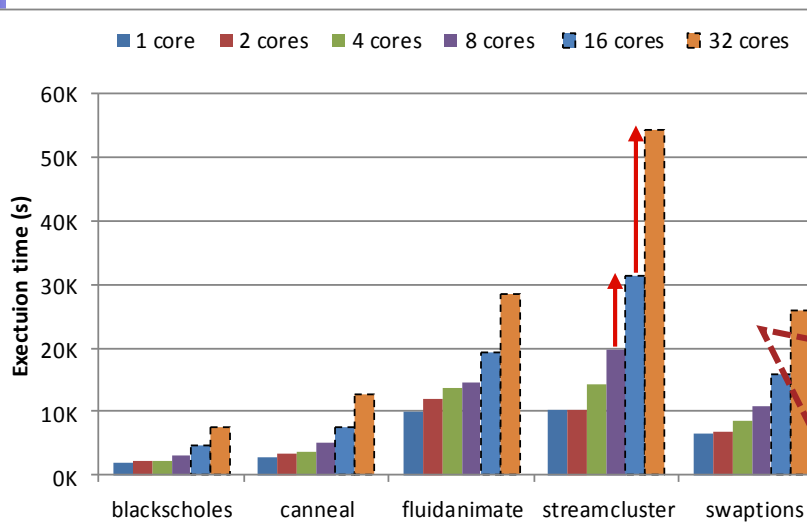
Sampling simulation

[DAC 14']DAPs: Dynamic Adjustment and Partial Sampling for Multithreaded/Multicore Simulation

Chien-Chih Chen, Yin-Chi Peng, Cheng-Fen Chen, Wei-Shan Wu, Qinghao Min, Pen-Chung Yew, Weihua Zhang, Tien-Fu Chen

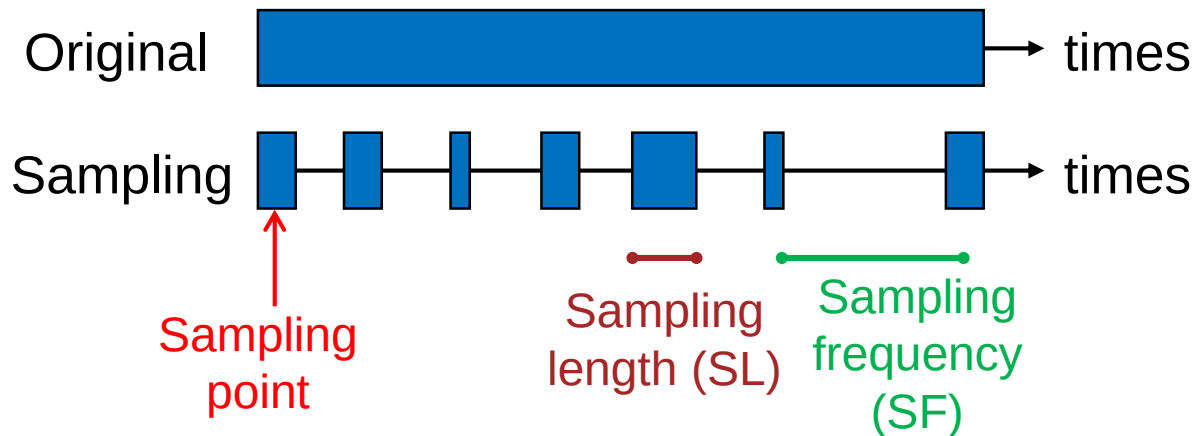
Timing model dominates simulation time

- Simulation can be decomposed into two models
 - Functional model (FM)
 - Detailed timing model (TM)
- Simulation time increases non-linearly as core no. increases
- Detailed TM dominates simulation time
- Statistical sampling can



Sampling Mechanism

- Simulate a **small representative** subset of program execution
- TM only executes on sampling point



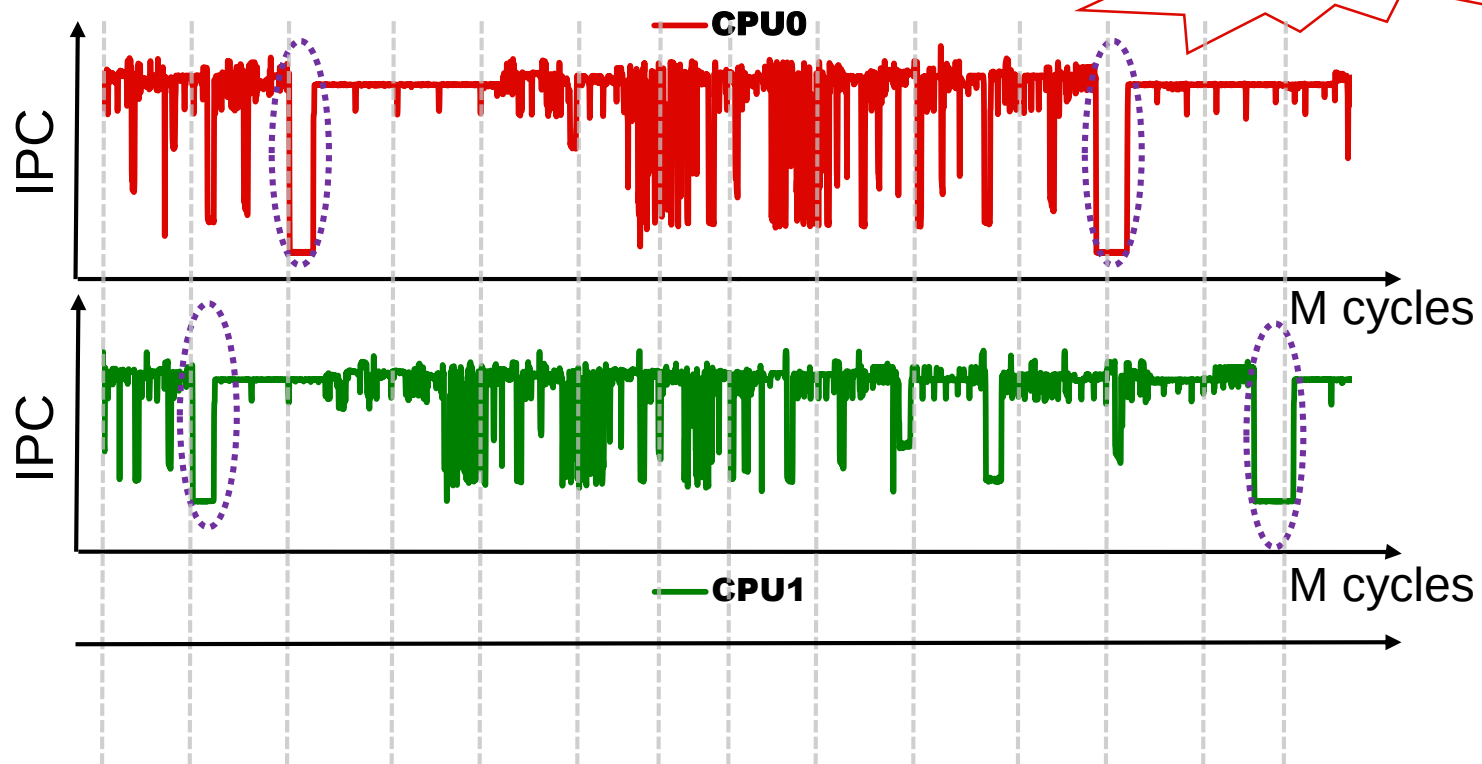
- Sampling techniques
 - Static sampling
 - Select regular sampling points before run-time
 - Dynamic sampling
 - Decide sampling points at run-time

Challenges in Multithreaded/Multicore simulation

- Synchronization events would cause simulation time to stall
 - Static sampling could not capture program variations
- In multithreaded application, threads don't run simultaneously
 - Synchronized core sampling cannot capture system events

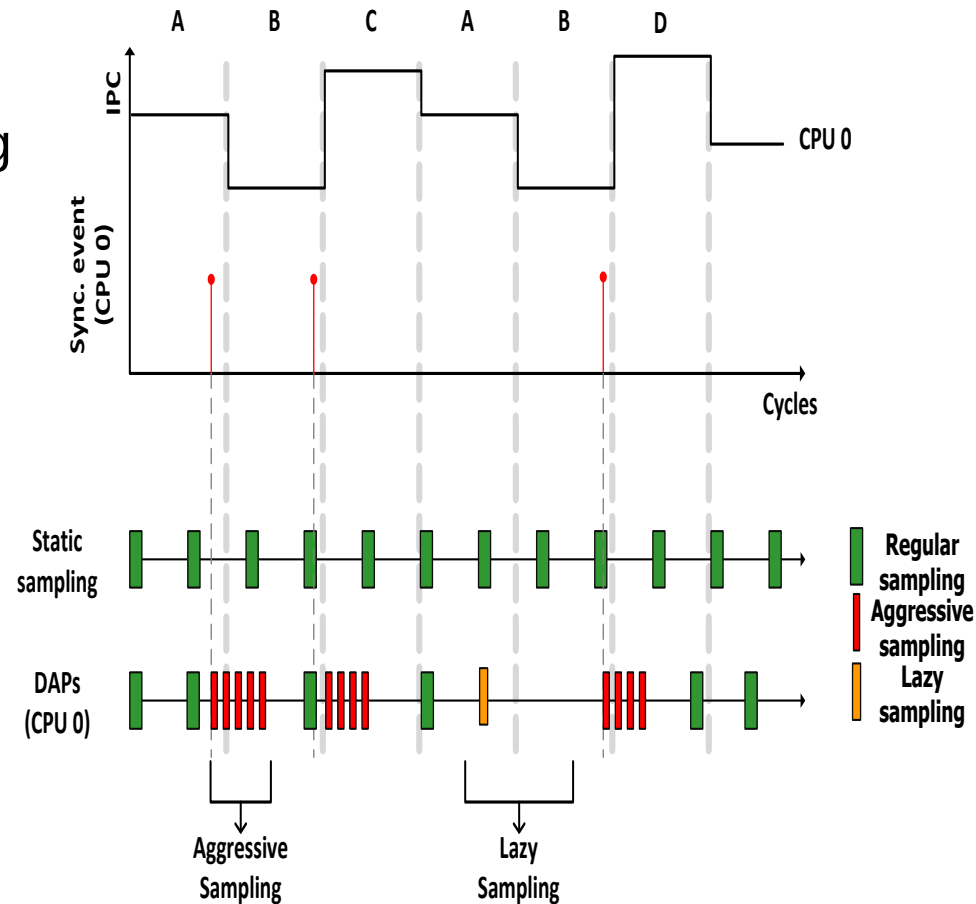
Dynamic sampling

Partial core sampling



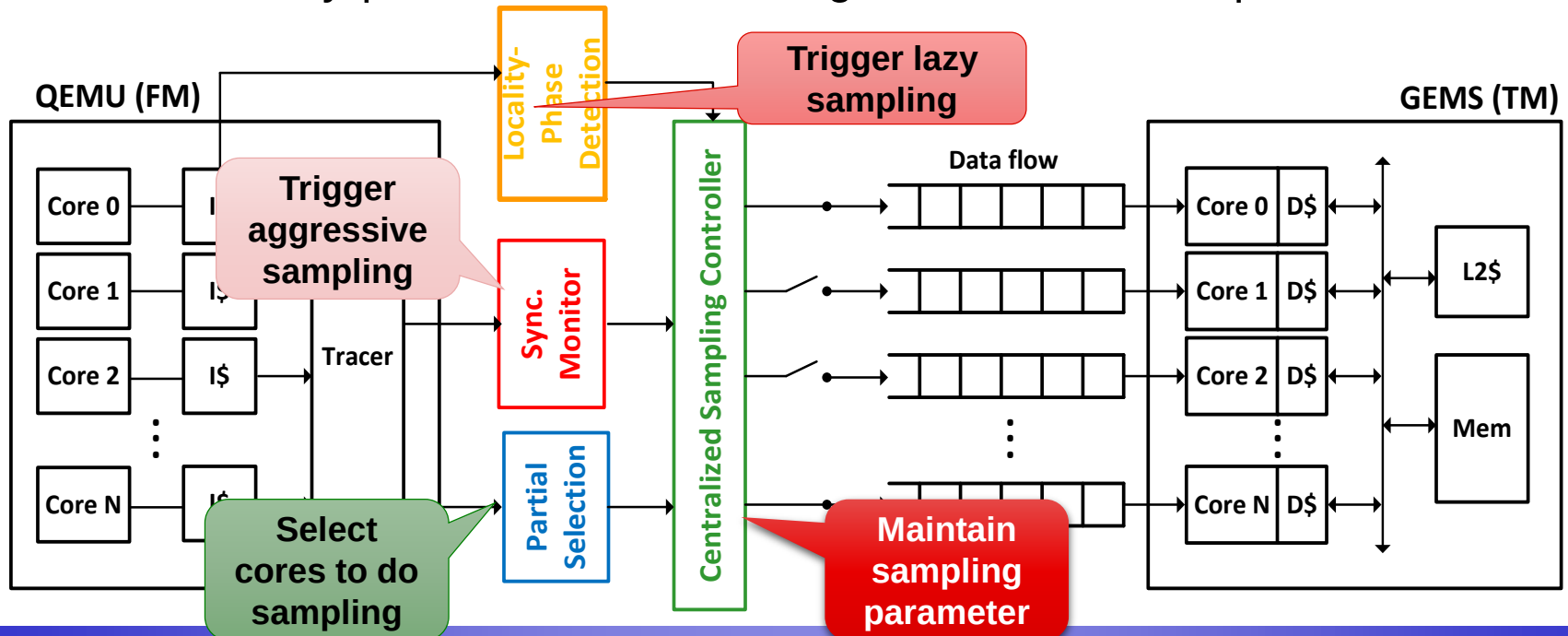
Hybrid sampling mechanisms for Multithreaded/Multicore simulation

- Regular sampling
 - ⊙ As the conventional sampling
- Aggressive sampling
 - ⊙ Capture thread synchronization primitives
 - ⊙ Frequently simulate to resist the variation
 - ⊙ Simulation accuracy ↑
- Lazy sampling
 - ⊙ Rarely simulate the repeated phases
 - ⊙ Simulation speed ↑



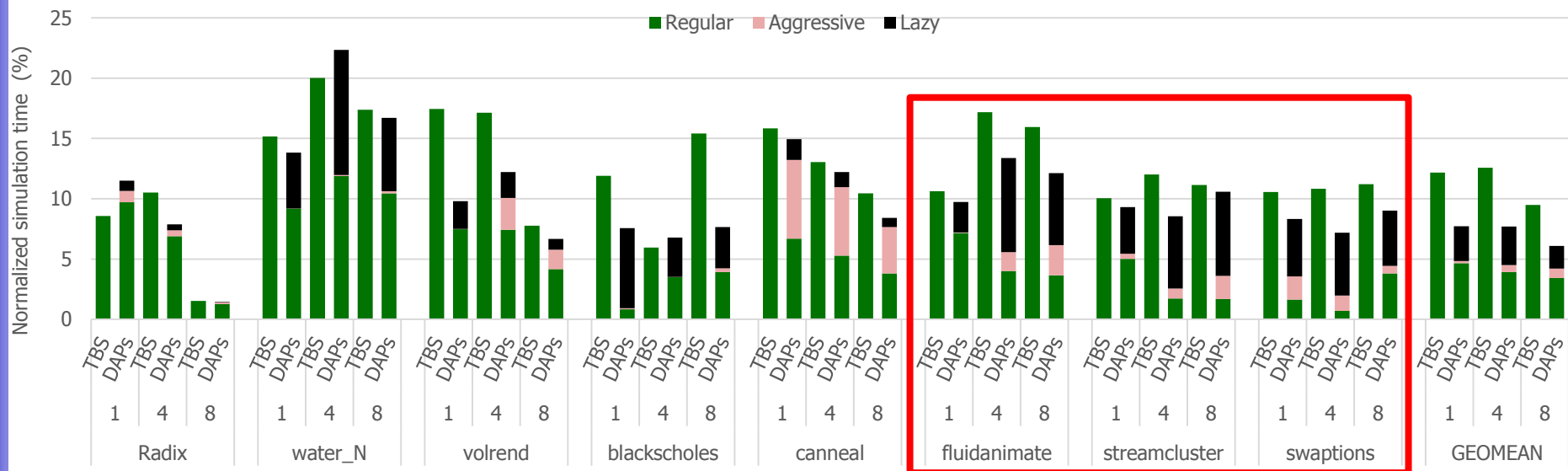
The DAPs framework

- Centralized sampling controller
 - Manage the sampling parameters and control the TM
- Synchronization monitor: capture synchronization primitives
- Partial selection: let each core has its own sampling switcher
- Locality-phase detection: distinguish the executed phase



Results: simulation speedup

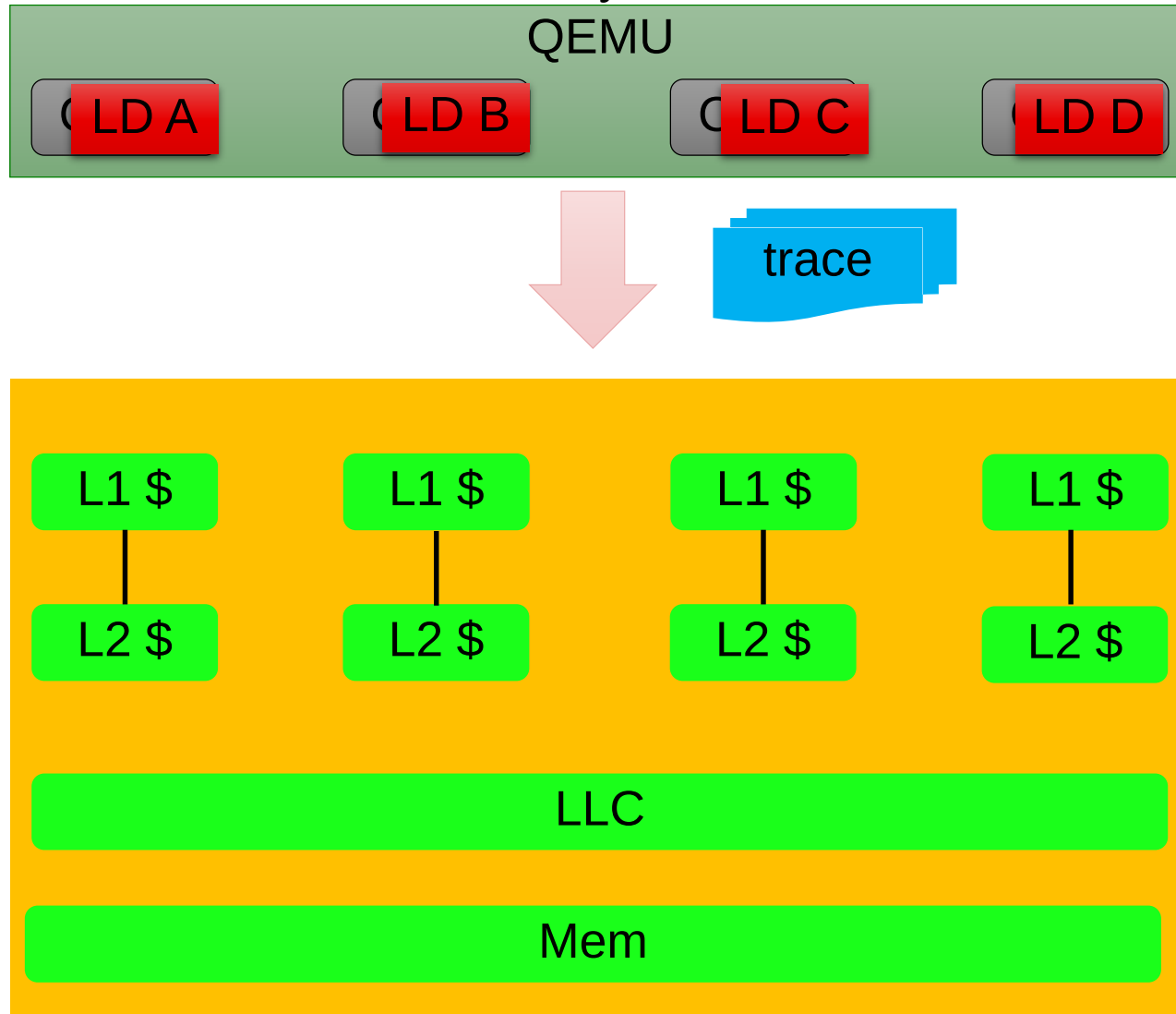
- Mark the lazy sampling for a high priority
 - Recover the simulation speed for the aggressive sampling
- More than 16x faster than fully-timing simulation.
- The simulation speed of DAPs is 1.5x~1.6x faster than TBS.
- In fluidanimate, streamcluster, swaptions
 - DAPs has a gain of simulation speed via lazy sampling.



Event-driven timing simulation w/ QEMU

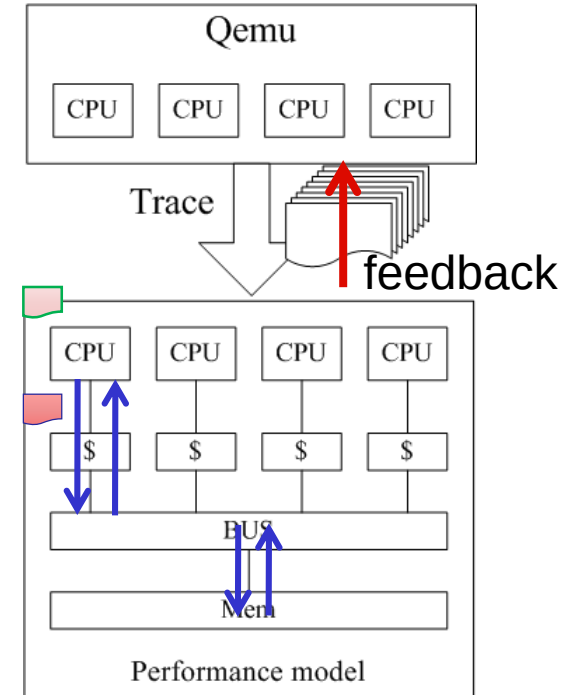
Challenges of trace-driven simulation

- Execution flow isn't affected by structural hazard.



Design Challenges

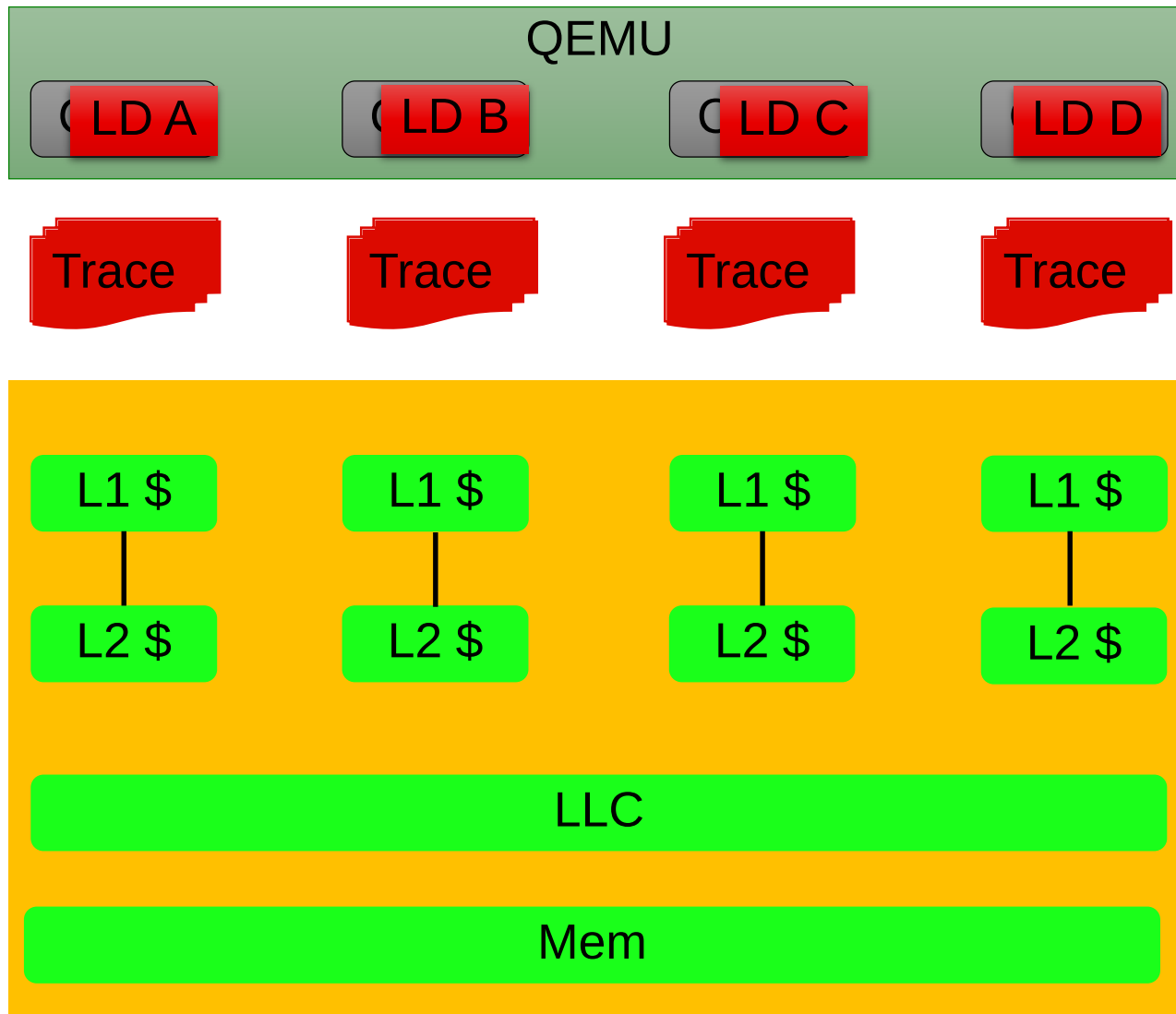
- Benefit of Qemu
 - Support full-system
 - Enough Simulation speed
 - Stability
- Difficulty of Qemu
 - Complex code translation
 - No support for performance model trace
 - Need to affect the progress of application
- Difficulty of Large-scale architecture simulation
 - Communicate between concurrent Qemu simulations
 - Synchronization overhead



Performance model requirement

Instruction count	Instruction address(memory access)
Data address (memory access)	Program counter
Multi-core simulation sequence	Performance model parallelism
Retrieve trace more effectively	Performance model flexibility

Skip the stalled core

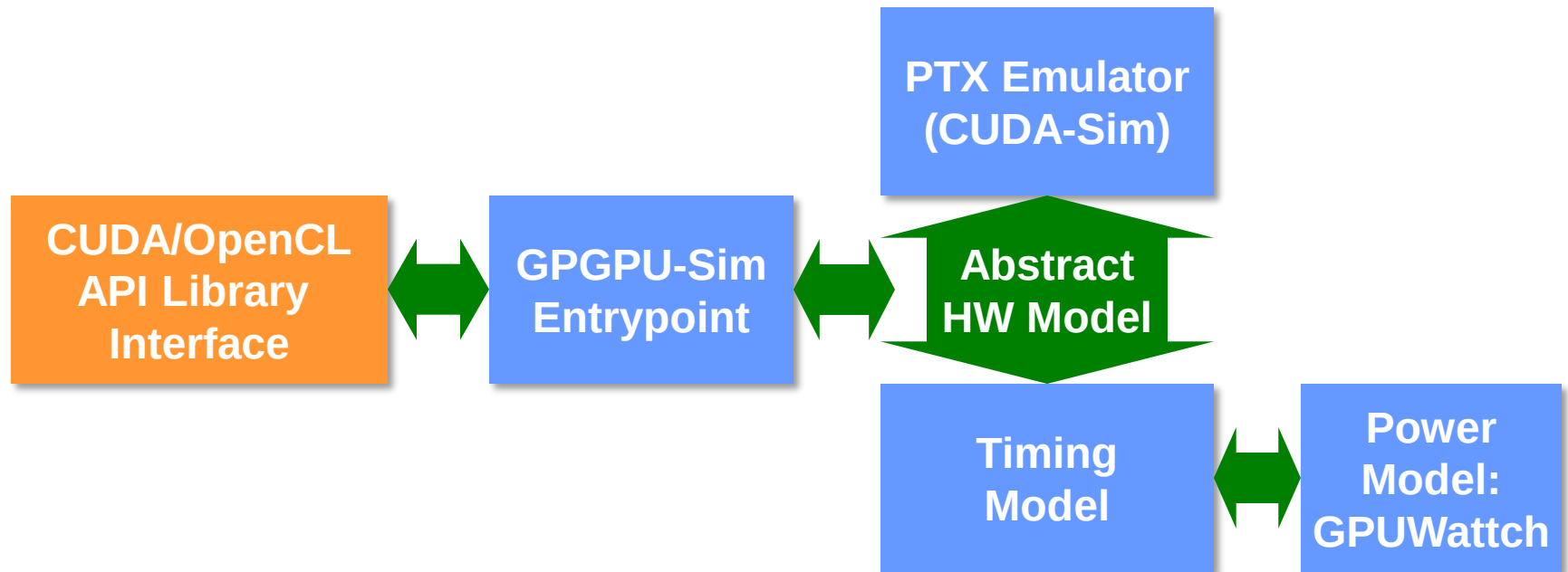


Cycle-Accurate HSA Simulator

Connect to `gpgpusim`

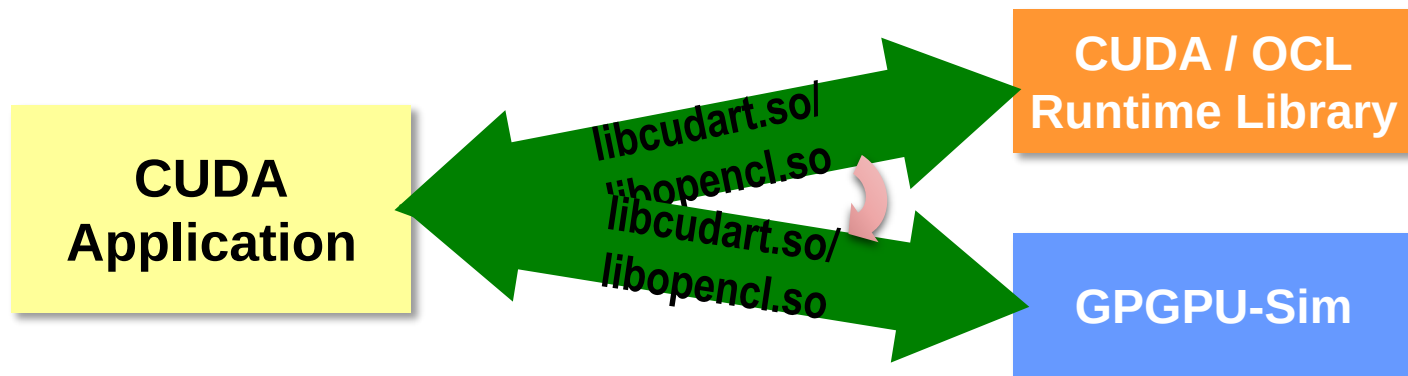
GPGPU-Sim Overview

- CUDA/OpenCL API library interface
- PTX instruction set emulator
- Timing model
- Power model

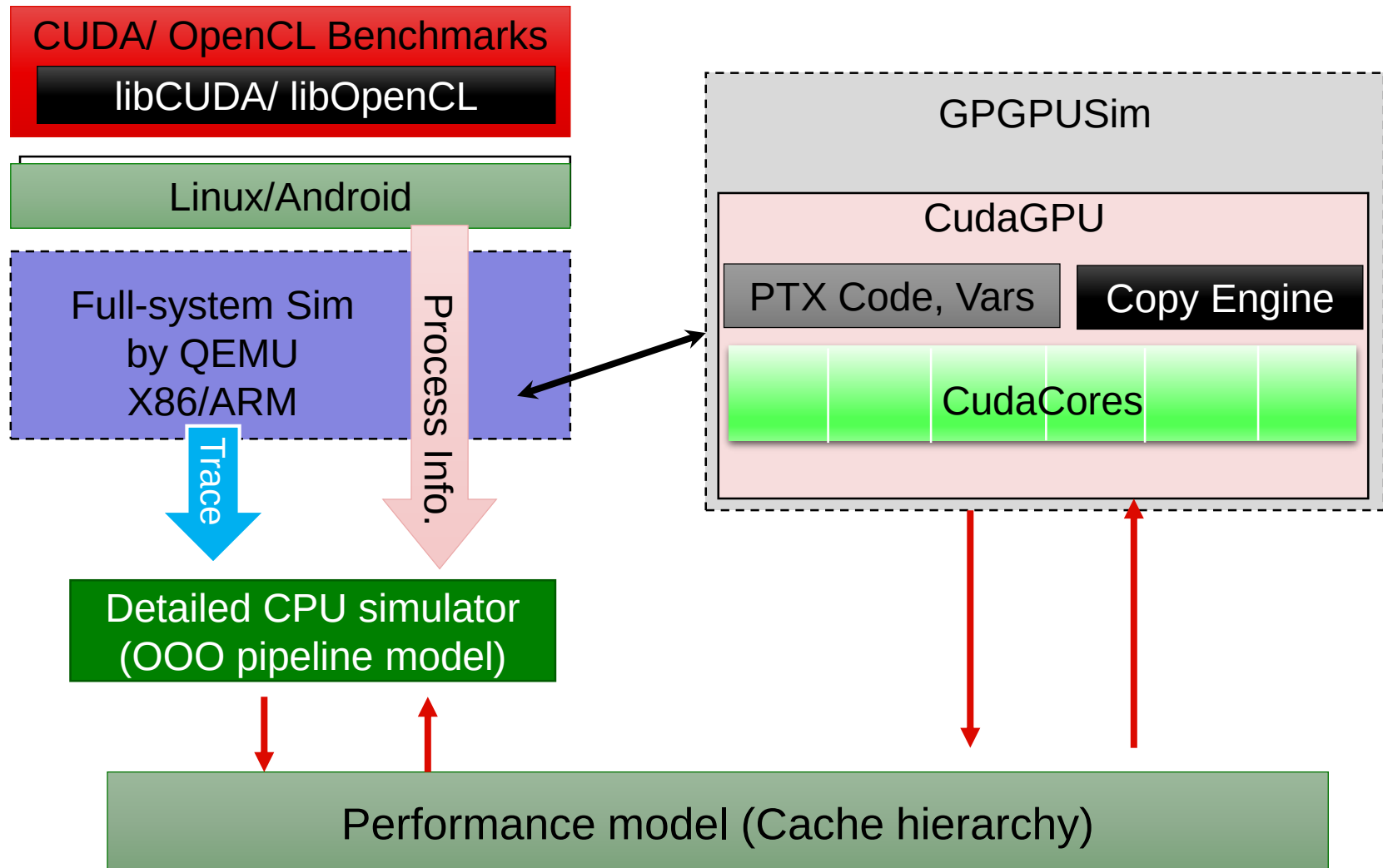


Interface to CUDA/OpenCL(OCL) API

- Implement OpenCL / CUDA interface calls in a new DLL.
- Adjust LD_LIBRARY_PATH and CUDA/OpenCL application runs on simulator rather than GPU hardware.



Building full-system memory exploration environment



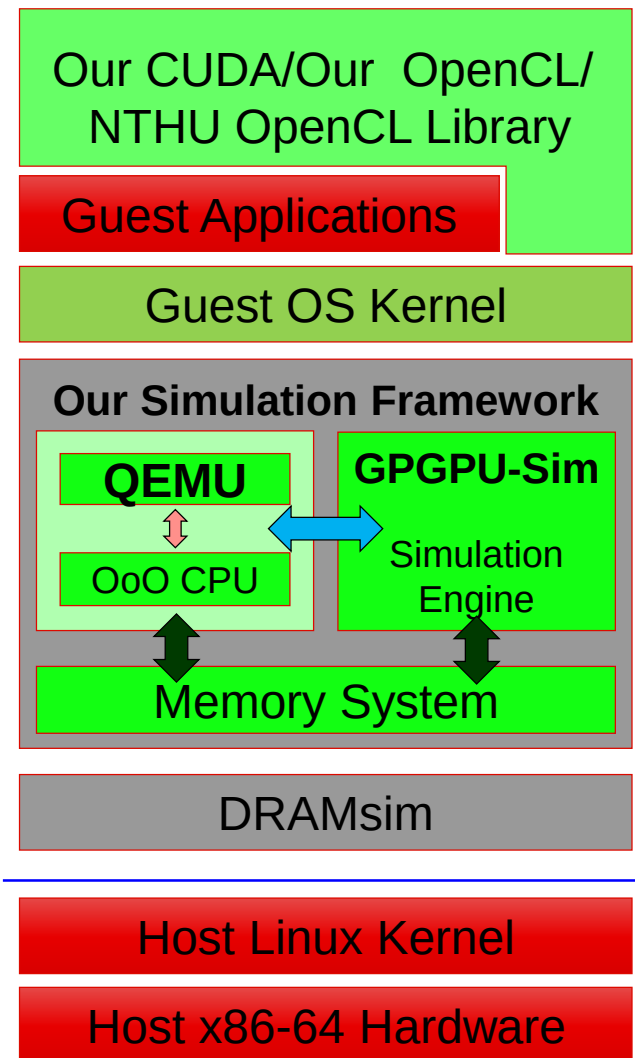
Comparison of Simulator Features

Name	X86 ISA	ARM ISA	Linux	Android	CPU model	GPU mode I	CUDA/OpenCL	Shared address space
QEMU	√	√	√	√				
MARSSx86	√		√	√	√			
Multi2Sim	√				√	√	OpenCL +(CUDA)	
GPGPU-Sim						√	OpenCL +CUDA	
HSAemu		√	√			√	OpenCL	
gem5-gpu	√	√	√	√	√	√	CUDA	√
Ours	√	√	√	√	√	√	OpenCL +CUDA	√

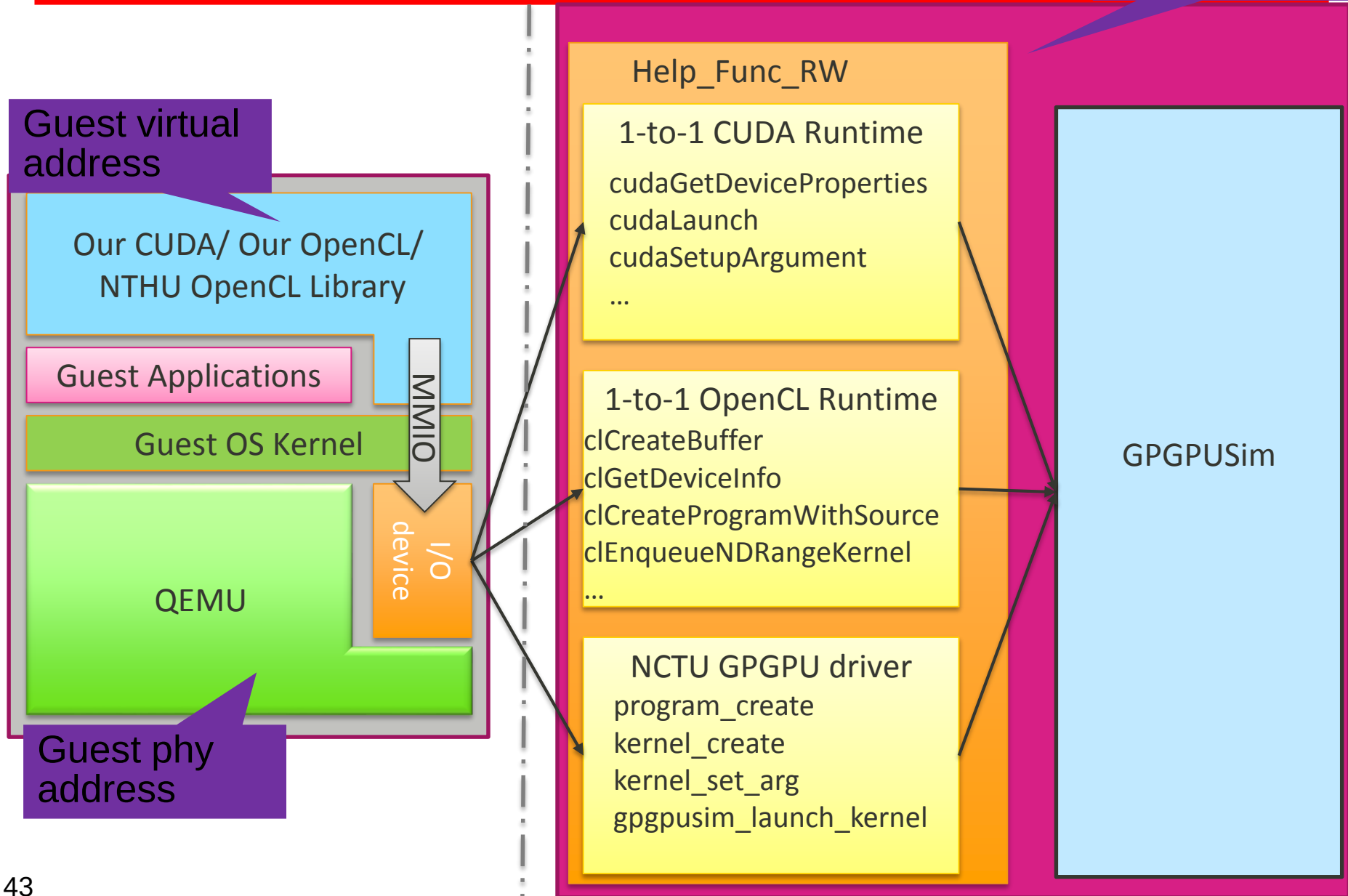
Our goals: Full system + Qemu + GPU

Proposed Full-System Simulation Framework for HSA and memory

- Integration of QEMU and GPGPU-Sim
- Simulation framework with
 - ⊙ Detailed CPU (fast), GPU simulation
 - ⊙ Shared address space via QEMU Mem
 - ⊙ Support CUDA and OpenCL
 - ⊙ Detailed memory systems (from M2S)
 - ⊙ Full-system supporting
 - ⊙ Vertical profiling from application and OS kernel



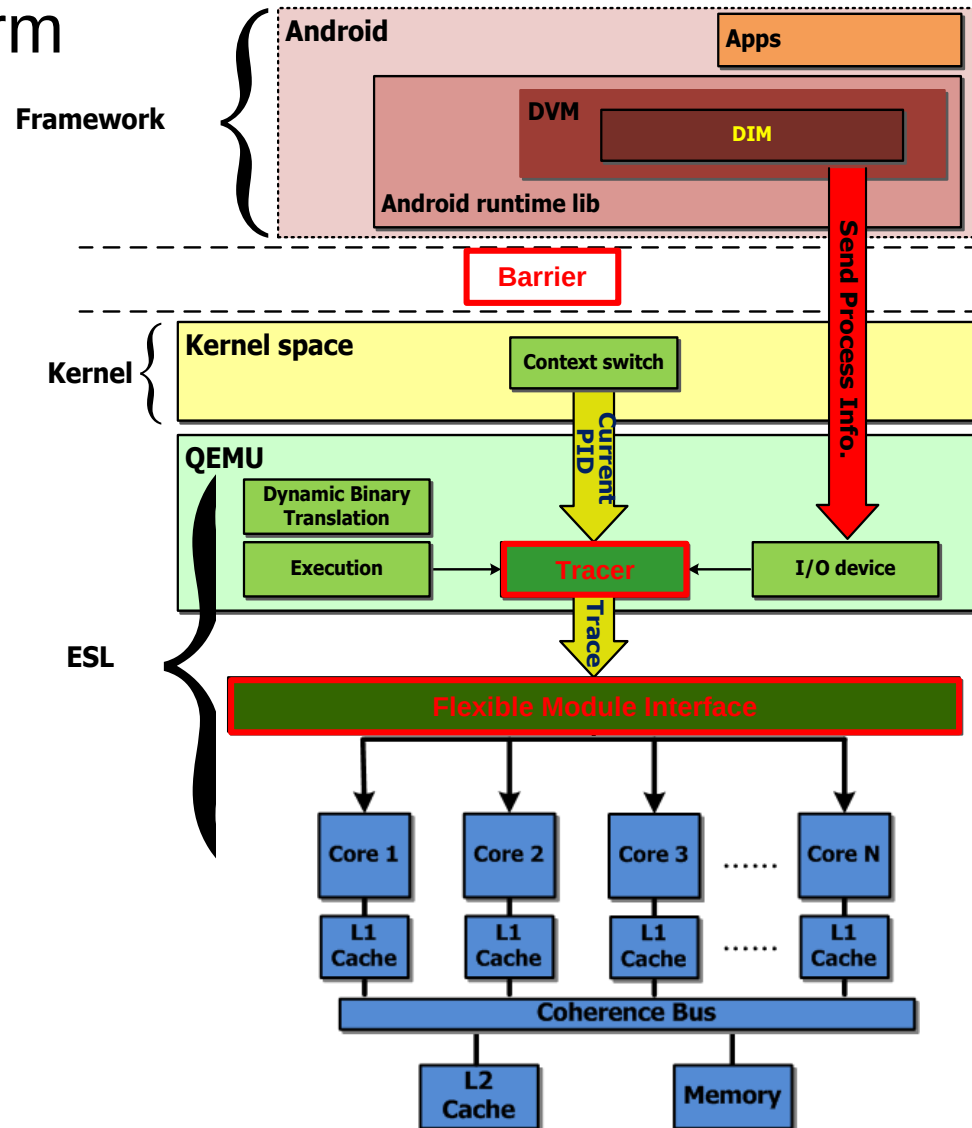
Handle multiple address domain



Full-system ESL and Architecture by Multi-granularity Android Tracer

- A Qemu-based on platform

- Full-system simulator for manycore and OS behavior
- Trace mechanism in Qemu
- Include instruction/data trace
- Exchanging info with performance model
- Flexible cycle accurate timing model



The End

Thank you!



Keep going ...